

JAVA OBJECT-ORIENTED PROGRAMMING

Listing 2 Analysis: Main.java (Car Demo) Memory Allocation Diagram

Exercise 2: Draw Stack and Heap In-front of Code. (Includes Instantiation, Null Construction, Reference, and Method Call tracking)

Listing 2 - Main.java (Car demo)

```
/*
 * File: Main.java
 * Purpose: Creates and controls Car objects
 */
public class Main {
    public static void main(String[] args) {
        // 1. Instantiation & Reference
        // Allocation
        Car bmw = new Car();
        Car tesla = new Car();

        // 2. State Mutators (Field Updates)
        bmw.setDetails("BMW", "Black");
        tesla.setDetails("Tesla", "Red");

        // 3. Method Invocation
        bmw.startEngine();
        tesla.startEngine();

        // 4. Display State
        bmw.displayState();
        tesla.displayState();

        // 5. State Change & Re-display
        bmw.stopEngine();
        bmw.displayState();
    }
}
```

Memory Allocation (Stack vs. Heap)

STACK MEMORY

Frame: main()

args: String[] (ref)

bmw: 0x101 Ref

tesla: 0x102 Ref

HEAP MEMORY

Car Object @ 0x101

- model = "BMW"
- color = "Black"
- isEngineOn = false

Car Object @ 0x102

- model = "Tesla"
- color = "Red"
- isEngineOn = true

Key Execution & Memory Concepts Breakdown

1

Instantiation

Car bmw = new Car(); allocates dynamic memory on the **Heap** for the new Car instance. Default field values are constructed (e.g., null for String objects, false for boolean primitive types).

2

Reference Variable

The local variable bmw lives inside the main() frame on the **Stack** and holds the memory address (e.g., 0x101) pointing directly to the heap instance.

3

Dot Operator (.) Method Call

Statements like bmw.setDetails(...) use the dot operator to dereference the address stored on the stack and manipulate instance fields stored in the heap object.